

Agentic AI Engineer Resume Preparation For Big Tech

Steps to make the resume big-tech ready

Step 1: Choose one target role

Use one primary title:

Agentic AI Engineer | Applied AI Engineer

Do not target five unrelated positions with the same resume.

Step 2: Study 10–15 job descriptions

Collect the repeated requirements:

- Python
- Data structures and algorithms
- System design
- LLM agents
- RAG
- Tool calling and MCP
- FastAPI
- Evaluation
- Docker and cloud
- Monitoring and reliability

Step 3: Build three strong projects

Use:

1. **AgentWriter AI**
2. **TripMate AI**
3. **Agent Guardrails and Evaluation Framework**

Each project should include:

- GitHub repository
- Detailed README
- Architecture diagram
- Demo video
- Tests
- Evaluation results
- Deployment link

Step 4: Add production engineering

Upgrade your projects with:

- Authentication
- PostgreSQL
- Async APIs
- Docker
- CI/CD
- Logging and tracing
- Retry and fallback logic
- Rate limiting
- Error handling
- Cloud deployment

Step 5: Create an evaluation dataset

Test each agent with at least **50–100 scenarios**.

Measure:

- Task-completion rate
- Tool-call success rate
- Structured-output success
- Hallucination rate
- Guardrail blocking rate
- Response latency
- Cost per request

Never invent results.

Step 6: Convert project descriptions into achievements

Weak:

Built a travel agent using LangGraph.

Strong:

Built an MCP-based travel-planning system integrating three external tools, achieving 92% successful tool execution across 100 evaluation scenarios.

Step 7: Prepare a one-page ATS resume

Recommended order:

1. Contact information
2. Summary
3. Technical skills
4. Professional experience
5. Agentic AI projects
6. Technical leadership
7. Education and publication

Avoid photos, graphics, rating bars, tables and unnecessary colors.

Step 8: Customize every application

For each job:

- Match your title to the position
- Move relevant skills higher
- Use important keywords from the job description
- Select the most relevant projects
- Rewrite your summary
- Remove unrelated information

Step 9: Improve your GitHub profile

Pin your best three repositories. Each should clearly show:

- Business problem
- Architecture
- Installation instructions

- API examples
- Evaluation approach
- Results
- Screenshots
- Limitations
- Future improvements

Step 10: Apply strategically

Use:

- Company career pages
- LinkedIn
- Employee referrals
- Recruiter outreach
- Open-source communities
- AI engineering events

YOUR NAME

Agentic AI Engineer | Applied AI Engineer | LLM Systems Engineer

Country| your@email.com | LinkedIn | GitHub | Portfolio | YouTube

SUMMARY

Agentic AI Engineer with experience building multi-agent applications, tool-using AI systems, RAG pipelines and production-ready LLM backends using Python, LangGraph, MCP, FastAPI, Groq and PostgreSQL. Experienced in agent orchestration, guardrails, human-in-the-loop workflows, evaluation, API development and real-time execution tracking. Strong engineering background in debugging, system design and complex technical problem-solving.

TECHNICAL SKILLS

Languages: Python, SQL, JavaScript

Agentic AI: LangGraph, MCP, Multi-Agent Systems, Supervisor Agents, Tool Calling, Human-in-the-Loop

LLM Systems: RAG, Embeddings, Vector Search, Structured Outputs, Prompt Engineering, Context Engineering

Backend: FastAPI, REST APIs, WebSockets, PostgreSQL, Pydantic, Async Programming

Reliability: Guardrails, Retry Logic, Fallbacks, Input Validation, Output Validation, Logging, Evaluation

DevOps: Docker, GitHub Actions, Cloud Deployment

Models: Groq, OpenAI, Gemini, Hugging Face

PROJECTS

AgentWriter AI — Multi-Agent Content Platform

LangGraph, Groq, FastAPI, PostgreSQL, WebSockets

- Architected a stateful multi-agent platform coordinating planning, research, writing and review agents through LangGraph subgraphs.
- Designed a supervisor agent to route tasks, manage agent transitions and recover from failed execution paths.
- Implemented guardrails and human approval checkpoints before publishing generated content.
- Built real-time workflow streaming to display agent planning, tool usage and completion status in the user interface.

- Developed FastAPI services and PostgreSQL storage for workflow states, generated content and execution history.
- Evaluated the system across **[X] test cases**, achieving **[X]% task-completion rate**, **[X]% valid structured-output rate** and **[X]-second p95 latency**.
- Deployed the application using Docker and **[cloud platform]**, with structured logging and API error monitoring.

GitHub: [Repository link] | **Demo:** [Live demo]

TripMate AI — MCP-Based Travel Planning System

MCP, LangGraph, Groq, FastAPI, External APIs

- Built an MCP-based travel assistant connecting flight, weather and destination services through standardized tool interfaces.
- Implemented supervisor-agent routing to select the correct specialist agent and external tool for each request.
- Designed structured tool schemas, input validation and response parsing for reliable API communication.
- Added retry policies, timeout handling and fallback responses for unavailable or malformed third-party API results.
- Introduced human-in-the-loop approval for high-impact recommendations and travel actions.
- Achieved **[X]% successful tool execution** across **[X] evaluation scenarios** while reducing failed workflows by **[X]%**.

GitHub: <https://github.com/entbappy/TripMate-AI-Using-MCP>

AI Agent Guardrails and Evaluation Framework

Python, Pydantic, LLM APIs, Safety Evaluation

- Developed a reusable guardrail layer for prompt injection, harmful inputs, restricted tool actions and malformed model outputs.
- Implemented structured-output validation, tool authorization policies and human escalation for high-risk requests.
- Created an adversarial evaluation dataset containing **[X] test cases** covering indirect prompt injection, unsafe actions and tool misuse.
- Measured guardrail precision, false-positive rate and unsafe-output blocking rate across multiple LLMs.
- Improved safe task completion from **[X]% to [X]%** while maintaining acceptable response latency.

GitHub: [Repository link]

PROFESSIONAL EXPERIENCE

Layout Engineer — Ulkasemi

[Location] | [Start Date] – [End Date or Present]

- Contributed to semiconductor layout development for advanced TSMC technology nodes using Cadence Virtuoso and Mentor Calibre.
- Performed physical verification, debugging and design-rule analysis for complex integrated-circuit layouts.
- Investigated layout violations and collaborated with engineering teams to deliver verified design solutions.
- Improved technical problem-solving through work involving strict reliability, accuracy and quality requirements.
- Documented design decisions and supported review workflows across cross-functional engineering teams.

OPEN-SOURCE AND TECHNICAL LEADERSHIP

Founder and Technical Educator — DSwithBappy

- Produce technical content on Agentic AI, LangGraph, MCP, RAG, FastAPI, guardrails and multi-agent systems.
- Built an audience of **[X]+ subscribers** and generated **[X]+ total views** through practical AI engineering tutorials.
- Published open-source projects demonstrating agent orchestration, tool integration and production LLM architectures.
- Delivered live technical sessions on building, evaluating and deploying AI agent systems.
- Helped developers learn production-focused Agentic AI through end-to-end project implementations.

YouTube: [Channel link]

EDUCATION

[Degree Name]

[University Name] | [Graduation Year]

PUBLICATION

“Electric Field-Induced Band Quenching and Semiconductor-to-Metal Transition in Bilayer α 12-Borophene Nanoribbons.”

- Conducted computational analysis and contributed to technical research, scientific interpretation and publication preparation.